

Rust Interview Codes

A practical collection of Rust programs
(structs, traits, closures, patterns, series & more)

Each section presents a problem and an idiomatic Rust solution, with sample output.

1. Sum of two numbers from user input

Reading two integers from standard input and printing their sum. Rust uses the `std::io` module for input — there is no shortcut like Python's `input()`.

```
use std::io::{self, Write};

fn read_int(prompt: &str) -> i64 {
    print!("{}", prompt);
    io::stdout().flush().unwrap();
    let mut buf = String::new();
    io::stdin().read_line(&mut buf).unwrap();
    buf.trim().parse().unwrap()
}

fn main() {
    let num1 = read_int("Enter 1st number: ");
    let num2 = read_int("Enter 2nd number: ");
    println!("num1 + num2 = {}", num1 + num2);
}
```

Output:

```
Enter 1st number: 3333
Enter 2nd number: 666
num1 + num2 = 3999
```

2. Find the GCD of two numbers

Greatest Common Divisor implemented recursively using the Euclidean algorithm. Rust uses pattern matching on the base cases for clarity.

```
fn gcd(a: u64, b: u64) -> u64 {
    if a == 0 || a == b { return b; }
    if b == 0 { return a; }
    if a > b { gcd(a - b, b) } else { gcd(a, b - a) }
}

fn main() {
    let (num1, num2) = (235u64, 875u64);
    println!("GCD = {}", gcd(num1, num2));
}
```

Output:

```
GCD = 5
```

3. Creating instance member fields (struct equivalent)

Rust has no classes. The closest equivalent is a **struct** with an `impl` block. All fields must be declared up-front in the struct definition — you cannot add new fields dynamically on a single instance like Python's `t1.c = 15`. A common idiom for optional / late-bound data is `Option<T>` or a `HashMap`.

```
use std::collections::HashMap;

struct Test {
    a: i32,
    b: i32,
    extra: HashMap<String, i32>, // for late-bound fields
}

impl Test {
    fn new() -> Self {
        Test { a: 5, b: 0, extra: HashMap::new() }
    }
    fn f1(&mut self) {
        self.b = 10;
    }
}

fn main() {
    let mut t1 = Test::new();
    let t2 = Test::new();

    t1.f1();
    t1.extra.insert("c".to_string(), 15); // "3rd instance variable"

    println!("t1: a={}, b={}, extra={:?}", t1.a, t1.b, t1.extra);
    println!("t2: a={}, b={}, extra={:?}", t2.a, t2.b, t2.extra);
}
```

Output:

```
t1: a=5, b=10, extra={"c": 15}
t2: a=5, b=0, extra={}
```

4. Addition using a closure (lambda equivalent)

Rust closures are the equivalent of Python lambdas. The `|args|` syntax replaces `lambda args: .`

```
fn main() {
    let f = |a: i32, b: i32| a + b;
    let r = f(3, 7);
    println!("{}", r);

    // Or inline:
    println!("{}", (|a: i32, b: i32| a + b)(3, 7));
}
```

Output:

```
10
10
```

5. Finding factorial using a closure

Rust closures cannot easily recurse by name, so the idiomatic approach is to use an iterator with `fold` or `product`.

```
fn main() {
    // Iterator-based factorial – feels closest to a one-liner lambda.
    let fact = |n: u64| (1..=n).product::<u64>();

    for n in [0u64, 1, 5, 7, 10] {
        println!("{}", n, fact(n));
    }
}
```

Output:

```
0! = 1
1! = 1
5! = 120
7! = 5040
10! = 3628800
```

6. Default arguments in functions

Rust does **not** support default function arguments directly. The common idioms are: (1) wrap the optional parameter in `Option<T>`, (2) use the **builder pattern**, or (3) write a wrapper function.

Example using `Option`:

```
fn add(a: i32, b: i32, c: Option<i32>) -> i32 {
    let c = c.unwrap_or(5); // default value
    a + b + c
}

fn main() {
    let s = add(2, 3, None);           // uses default 5
    println!("The addition is: {}", s);

    let s2 = add(2, 3, Some(10));     // override
    println!("With override:   {}", s2);
}
```

Output:

```
The addition is: 10
With override:   15
```

7. Extract integer values from a heterogeneous collection

Rust is statically typed, so a vector cannot directly hold values of different types. The Rust way is to define an **enum** to wrap each variant and then filter for the one we want.

```
#[derive(Debug, Clone)]
enum Item {
    Int(i64),
    Float(f64),
    Complex(f64, f64),
    Str(String),
}

fn main() {
    let l1: Vec<Item> = vec![
        Item::Str("abc".into()),
        Item::Float(34.56),
        Item::Int(32),
        Item::Complex(3.0, 4.0),
        Item::Str("b".into()),
        Item::Int(55),
        Item::Float(65.7),
        Item::Str("90".into()),
        Item::Int(180),
    ];

    let l2: Vec<i64> = l1.iter().filter_map(|e| {
        if let Item::Int(v) = e { Some(*v) } else { None }
    }).collect();

    println!("{:?}", l1);
    println!("{:?}", l2);
}
```

Output:

```
[Str("abc"), Float(34.56), Int(32), Complex(3.0, 4.0), Str("b"), Int(55), Float(65.7), Str("90"), Int(180)]
[32, 55, 180]
```

8. Does Rust support multiple inheritance?

Rust has **no inheritance** in the OOP sense. Instead, it uses **traits**, and a single type can implement many traits — giving the composition benefit of multiple inheritance without its pitfalls.

```
trait Walker {
    fn walk(&self) {
        println!("walking...");
    }
}

trait Swimmer {
    fn swim(&self) {
        println!("swimming...");
    }
}
```

```

    }
}

struct Duck;

impl Walker for Duck {}
impl Swimmer for Duck {}

fn main() {
    let d = Duck;
    d.walk();
    d.swim();
}

```

Output:

```

walking...
swimming...

```

9. Print a right-angled star triangle

A classic nested-loop pattern. Rust uses ranges and `print !` without a newline.

```

fn main() {
    let s: u32 = 5;
    for i in 1..=s {
        for _ in 0..i {
            print!("* ");
        }
        println!();
    }
}

```

Output:

```

*
* *
* * *
* * * *
* * * * *

```

10. Number pattern: 1 / 1 2 / 1 2 3 ...

Each row prints numbers starting from 1.

```

fn main() {
    let s: u32 = 5;
    for i in 1..=s {
        for j in 1..=i {
            print!("{}", j);
        }
        println!();
    }
}

```

```
}  
}
```

Output:

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```

11. Continuous number triangle: 1 / 2 3 / 4 5 6 / 7 8 9 10 / 11 12 13 14 15

A single counter num increments across all rows.

```
fn tri(n: u32) {  
    let mut num = 1;  
    for i in 0..n {  
        for _ in 0..=i {  
            print!("{}", num);  
            num += 1;  
        }  
        println!();  
    }  
}  
  
fn main() {  
    tri(5);  
}
```

Output:

```
1  
2 3  
4 5 6  
7 8 9 10  
11 12 13 14 15
```

12. Letter pattern: A / B B / C C C / D D D D / E E E E E

Use `char::from_u32` or simple cast from `u8` to print ASCII letters (65 = 'A').

```
fn main() {  
    let n: u32 = 5;  
    let mut p: u8 = b'A'; // 65  
    for i in 0..n {  
        for _ in 0..=i {  
            print!("{}", p as char);  
        }  
        println!();  
        p = p.wrapping_add(1);  
    }  
}
```

```

        p += 1;
    }
}

```

Output:

```

A
B B
C C C
D D D D
E E E E E

```

13. Letter pattern: A / B C / D E F / G H I J / K L M N O

Letters increment continuously across the whole pattern.

```

fn main() {
    let n: u32 = 5;
    let mut p: u8 = b'A';
    for i in 0..n {
        for _ in 0..=i {
            print!("{}", p as char);
            p += 1;
        }
        println!();
    }
}

```

Output:

```

A
B C
D E F
G H I J
K L M N O

```

14. Letter pattern: A / A B / A B C / A B C D / A B C D E

Each row starts again from 'A'.

```

fn main() {
    let n: u32 = 5;
    for i in 0..n {
        let mut p: u8 = b'A';
        for _ in 0..=i {
            print!("{}", p as char);
            p += 1;
        }
        println!();
    }
}

```


Output:

```
A
A B
A B C
A B C D
A B C D E
```

15. Centered letter pattern: A / B C / D E F / G H I J / K L M N O

Padded with leading spaces to center the pyramid.

```
fn main() {
    let n: u32 = 5;
    let mut p: u8 = b'A';
    for i in 0..n {
        for _ in i..n {
            print!(" ");
        }
        for _ in 0..=i {
            print!("{}", p as char);
            p += 1;
        }
        println!();
    }
}
```

Output:

```
  A
 B C
D E F
G H I J
K L M N O
```

16. Centered alphabet pyramid: A / A B A / A B C B A / ...

Each row builds up to a peak then mirrors back to 'A'.

```
fn main() {
    let n: u32 = 5;
    for i in 0..n {
        for _ in i..n { print!(" "); }
        let mut p: u8 = b'A';
        // ascending
        for _ in 0..=i {
            print!("{}", p as char);
            p += 1;
        }
        // descending
        p -= 2;
        for _ in 0..i {
```

```

        print!("{}", p as char);
        p -= 1;
    }
    println!();
}
}

```

Output:

```

    A
  A B A
A B C B A
A B C D C B A
A B C D E D C B A

```

17. Reverse number pattern: 5 4 3 2 1 / 4 3 2 1 / 3 2 1 / 2 1 / 1

Each row drops the leading number.

```

fn main() {
    let n: i32 = 5;
    for i in 0..n {
        let mut k = n - i;
        for _ in 0..i { print!(" "); } // leading spaces for shift
        for _ in i..n {
            print!("{}", k);
            k -= 1;
        }
        println!();
    }
}

```

Output:

```

5 4 3 2 1
 4 3 2 1
  3 2 1
   2 1
    1

```

18. Diamond pattern of repeating numbers

Top half increases (1 / 2 2 2 / 3 3 3 3 ...), bottom mirrors back down.

```

fn main() {
    let n: i32 = 5;
    let mut p = 1;
    // Top half
    for i in 0..n {
        for _ in i..n { print!(" "); }
        for _ in 0..=i { print!("{}", p); }
        for _ in 0..i { print!("{}", p); }
    }
}

```

```

        p += 1;
        println!();
    }
    // Bottom half
    p -= 2;
    for i in 0..n - 1 {
        for _ in 0..i { print!(" "); }
        for _ in i + 1..n { print!("{}", p); }
        for _ in i + 1..n - 1 { print!("{}", p); }
        p -= 1;
        println!();
    }
}

```

Output:

```

    1
  2 2 2
3 3 3 3 3
4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5
4 4 4 4 4 4 4 4
3 3 3 3 3
2 2 2
1

```

19. Letter pattern with skip: A / C C / E E E / G G G G / I I I I I

Increase the letter by 2 each row (skip the odd letters).

```

fn main() {
    let n: u32 = 5;
    let mut p: u8 = b'A';
    for i in 0..n {
        for _ in 0..i {
            print!("{}", p as char);
        }
        p += 2;
        println!();
    }
}

```

Output:

```

A
C C
E E E
G G G G
I I I I I

```

20. Check whether a number is a Krishna Murthy (Strong) number

A Krishna Murthy number equals the sum of the factorials of its digits (e.g. $145 = 1! + 4! + 5!$). We extract digits with a `while` loop.

```
fn factorial(d: u64) -> u64 {
    (1..=d.max(1)).product::
```

Output:

```
145 -> Yes
1 -> Yes
2 -> Yes
40585 -> Yes
123 -> No
```

21. Find all factors of a number

Iterate from 1 to n and print every divisor.

```
fn main() {
    let n: u32 = 45;
    print!("Factors of {}: ", n);
    for i in 1..=n {
        if n % i == 0 {
            print!("{}", i);
        }
    }
    println!();
}
```

Output:

```
Factors of 45: 1 3 5 9 15 45
```

22. Check whether a number is prime

Trial division up to $\sqrt{n}$ is sufficient.

```
fn is_prime(n: u64) -> bool {
    if n < 2 { return false; }
    if n == 2 { return true; }
    if n % 2 == 0 { return false; }
    let mut i = 3;
    while i * i <= n {
        if n % i == 0 { return false; }
        i += 2;
    }
    true
}

fn main() {
    for n in [1u64, 2, 3, 4, 17, 25, 97, 100] {
        println!("{}", n, is_prime(n));
    }
}
```

Output:

```
1 is prime? false
2 is prime? true
3 is prime? true
4 is prime? false
17 is prime? true
25 is prime? false
97 is prime? true
100 is prime? false
```

23. Check if a number is Abundant or Deficient

Sum of proper divisors $> n$ -> abundant; otherwise deficient (or perfect when equal).

```
fn classify(n: u64) -> &'static str {
    let mut sum: u64 = 0;
    for i in 1..n {
        if n % i == 0 { sum += i; }
    }
    if sum > n { "Abundant number" }
    else if sum == n { "Perfect number" }
    else { "Deficient number" }
}

fn main() {
    for n in [12u64, 28, 34, 6, 100] {
        println!("{}", n, classify(n));
    }
}
```

```
}
```

Output:

```
12 -> Abundant number
28 -> Perfect number
34 -> Deficient number
6 -> Perfect number
100 -> Abundant number
```

24. Check if a number is Pronic

A pronic number is the product of two consecutive integers, e.g. $56 = 7 \times 8$.

```
fn is_pronic(n: u64) -> Option<(u64, u64)> {
    for i in 1..=n {
        if i * (i + 1) == n {
            return Some((i, i + 1));
        }
        if i * (i + 1) > n { break; }
    }
    None
}

fn main() {
    let n: u64 = 56;
    match is_pronic(n) {
        Some((a, b)) => println!("{}", a * b == n ? "Pronic number" : "Not a Pronic number"),
        None          => println!("{}", "Not a Pronic number"),
    }
}
```

Output:

```
7 x 8 = 56
Pronic number
```

25. Arithmetic series: $1 + 2 + 3 + \dots + N$

Iterators make this a one-liner with `sum()`.

```
fn main() {
    let n: u64 = 20;
    let total: u64 = (1..=n).sum();
    println!("Sum of series: {}", total);
}
```

Output:

```
Sum of series: 210
```

26. Arithmetic series with common difference: $9 + 13 + 17 + \dots$ (N terms)

Start at 9, step by 4, sum first N terms.

```
fn main() {
    let n: u64 = 20;
    let mut a: u64 = 9;
    let mut sum: u64 = 0;
    for _ in 0..n {
        sum += a;
        a += 4;
    }
    println!("Sum of series: {}", sum);
}
```

Output:

```
Sum of series: 940
```

27. Sum of first N even numbers: $2 + 4 + 6 + \dots + 2N$

```
fn main() {
    let n: u64 = 20;
    let sum: u64 = (1..=n).map(|i| 2 * i).sum();
    println!("Sum of series: {}", sum);
}
```

Output:

```
Sum of series: 420
```

28. Sum of first N odd numbers: $1 + 3 + 5 + \dots$

```
fn main() {
    let n: u64 = 20;
    let sum: u64 = (1..=n).map(|i| 2 * i - 1).sum();
    println!("Sum of series: {}", sum);
}
```

Output:

```
Sum of series: 400
```

29. Decreasing arithmetic series: $10 + 9 + 8 + \dots$ (N terms)

```
fn main() {
    let n: i64 = 10;
    let mut a: i64 = 10;
    let mut sum: i64 = 0;
    for _ in 0..n {
        sum += a;
        a -= 1;
    }
    println!("Sum of series: {}", sum);
}
```

Output:

```
Sum of series: 55
```

30. Power series: $x^1 + x^2 + x^3 + \dots + x^N$

Rust has pow on integers.

```
fn main() {
    let n: u32 = 10;
    let x: u64 = 3;
    let sum: u64 = (1..=n).map(|i| x.pow(i)).sum();
    println!("Sum of series: {}", sum);
}
```

Output:

```
Sum of series: 88572
```

31. Geometric series: $2 + 4 + 8 + 16 + \dots$ (N terms)

Each term is the previous one multiplied by 2.

```
fn main() {
    let n: u32 = 20;
    let mut a: u64 = 2;
    let mut sum: u64 = 0;
    for _ in 0..n {
        sum += a;
        a *= 2;
    }
    println!("Sum of series: {}", sum);
}
```

Output:

```
Sum of series: 4194300
```


32. Geometric series: $2 + 6 + 18 + 54 + \dots$ (N terms)

Common ratio = 3.

```
fn main() {
    let n: u32 = 20;
    let mut a: u64 = 2;
    let mut sum: u64 = 0;
    for _ in 0..n {
        sum += a;
        a *= 3;
    }
    println!("Sum of series: {}", sum);
}
```

Output:

```
Sum of series: 3486784400
```

33. Geometric series: $5 + 25 + 125 + \dots$ (N terms)

Common ratio = 5.

```
fn main() {
    let n: u32 = 10;
    let mut a: u128 = 5;
    let mut sum: u128 = 0;
    for _ in 0..n {
        sum += a;
        a *= 5;
    }
    println!("Sum of series: {}", sum);
}
```

Output:

```
Sum of series: 12207030
```

34. Alternating geometric series: $2 - 6 + 18 - 54 + \dots$

Toggle the sign on each iteration.

```
fn main() {
    let n: u32 = 20;
    let mut a: i128 = 2;
    let mut sum: i128 = 0;
    for i in 1..=n {
        if i % 2 == 0 { sum -= a; } else { sum += a; }
        a *= 3;
    }
    println!("Sum of series: {}", sum);
}
```

```
}
```

Output:

```
Sum of series: -1743392200
```

35. Operator overloading — implementing the Add trait

Rust overloads operators by implementing traits from `std::ops`. Add for `+`, Mul for `*`, Display for printable form.

```
use std::ops::{Add, Mul};
use std::fmt;

#[derive(Debug, Clone, Copy)]
struct Point { x: i32, y: i32 }

impl Add for Point {
    type Output = Point;
    fn add(self, other: Point) -> Point {
        Point { x: self.x + other.x, y: self.y + other.y }
    }
}

// Point * Point (element-wise)
impl Mul for Point {
    type Output = Point;
    fn mul(self, other: Point) -> Point {
        Point { x: self.x * other.x, y: self.y * other.y }
    }
}

// Point * scalar
impl Mul<i32> for Point {
    type Output = Point;
    fn mul(self, k: i32) -> Point {
        Point { x: self.x * k, y: self.y * k }
    }
}

impl fmt::Display for Point {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{}{}", self.x, self.y)
    }
}

fn main() {
    let p1 = Point { x: 2, y: 3 };
    let p2 = Point { x: 3, y: 4 };

    println!("{}", p1 + p2); // 5,7
    println!("{}", p1 * p2); // 6,12
    println!("{}", p1 * 3); // 6,9
}
```

Output:

```
5,7
6,12
6,9
```

36. Variadic-style and keyword-style parameters

Rust does not have variadic functions or keyword arguments in user code. The idioms are: (1) take a `&[T]` slice or `Vec<T>` for a variable number of values, and (2) take a `HashMap` for keyword-like input.

```
use std::collections::HashMap;

fn print_params(x: i32, y: i32, z: i32,
               pospar: &[i32],
               keypar: &HashMap<&str, i32>) {
    println!("{}", x, y, z);
    println!("{}", pospar);
    println!("{}", keypar);
}

fn main() {
    let mut kw = HashMap::new();
    kw.insert("foo", 1);
    kw.insert("bar", 2);

    print_params(1, 2, 3, &[5, 6, 7], &kw);

    let empty: HashMap<&str, i32> = HashMap::new();
    print_params(1, 2, 3, &[], &empty);
}
```

Output:

```
1 2 3
[5, 6, 7]
{"foo": 1, "bar": 2}
1 2 3
[]
{}
```

37. Print an N x N matrix of stars

Nested loops — a useful primer for the row/column index patterns that follow.

```
fn main() {
    let n: u32 = 9;
    for _ in 0..n {
        for _ in 0..n {
            print!("* ");
        }
        println!();
    }
}
```

```
}
}
```

Output:

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

38. Print the increasing pyramid: "C / C O / C O D / C O D E / C O D E R"

Walk through the characters of a string by index.

```
fn main() {
    let s = "CODER";
    let bytes = s.as_bytes();
    let n = bytes.len();
    for i in 0..n {
        for j in 0..=i {
            print!("{}", bytes[j] as char);
        }
        println!();
    }
}
```

Output:

```
C
C O
C O D
C O D E
C O D E R
```

39. Reverse string pyramid: "R / E E / D D D / O O O O / C C C C C"

Walk backwards through the string, repeating the current letter $i+1$ times.

```
fn main() {
    let s = "CODER";
    let bytes = s.as_bytes();
    let n = bytes.len();
    let mut p = n - 1; // start at 'R'
    for i in 0..n {
        for _ in 0..=i {
```

```

        print!("{}", bytes[p] as char);
    }
    if p > 0 { p -= 1; }
    println!();
}
}

```

Output:

```

R
E E
D D D
O O O O
C C C C C

```

40. Right-aligned pyramid: "R E D O C / E D O C / D O C / O C / C"

Pad each row with leading spaces so it right-aligns.

```

fn main() {
    let s = "CODER";
    let bytes = s.as_bytes();
    let n = bytes.len();
    let mut k = n - 1;
    for i in 0..n {
        // leading spaces (2 per column)
        for _ in 0..i { print!(" "); }
        let mut p = k;
        for _ in i..n {
            print!("{}", bytes[p] as char);
            if p == 0 { break; }
            p -= 1;
        }
        println!();
        if k > 0 { k -= 1; }
    }
}

```

Output:

```

R E D O C
 E D O C
  D O C
   O C
    C

```

41. Check whether two numbers share the same digits (in any order)

Extract each digit, sort, and compare.

```
fn digits(mut n: u64) -> Vec<u8> {
    if n == 0 { return vec![0]; }
    let mut d = Vec::new();
    while n > 0 {
        d.push((n % 10) as u8);
        n /= 10;
    }
    d.sort();
    d
}

fn main() {
    let (a, b): (u64, u64) = (123, 321);
    println!("{}", if digits(a) == digits(b) { "Yes" } else { "No" });

    let (a, b): (u64, u64) = (123, 124);
    println!("{}", if digits(a) == digits(b) { "Yes" } else { "No" });
}
```

Output:

```
Yes
No
```

42. Decode a numeric string to characters using ASCII

Read the reversed string in chunks of 2 or 3 digits depending on whether the value falls in the ASCII letter ranges (A-Z = 65-90, a-z = 97-122) or is a space (32).

```
fn decode(input: &str) -> String {
    // Step 1: reverse the digit string
    let s: Vec<char> = input.chars().rev().collect();
    let n = s.len();
    let mut res = String::new();
    let mut i = 0;
    while i + 1 < n {
        let two: String = s[i..i + 2].iter().collect();
        let two_val: u32 = two.parse().unwrap_or(0);

        if two == "32" {
            res.push(' ');
            i += 2;
        } else if (65..=90).contains(&two_val) || (97..=122).contains(&two_val) {
            res.push(char::from_u32(two_val).unwrap());
            i += 2;
        } else if i + 2 < n {
            let three: String = s[i..i + 3].iter().collect();
            let v: u32 = three.parse().unwrap_or(0);
            if let Some(c) = char::from_u32(v) {
                res.push(c);
            }
            i += 3;
        } else {
            break;
        }
    }
}
```

```

    }
    }
    res
}

fn main() {
    let input = "79611511011372111014110 8";
    let reversed: String = input.chars().rev().collect();
    println!("{}", reversed);
    println!("{}", decode(input));
}

```

Output:

```

8 0111411112731101151167
PrepInsta

```

43. Sum of digits of a number

Repeatedly take modulo 10 and divide by 10.

```

fn digit_sum(mut n: u64) -> u64 {
    let mut s = 0;
    while n > 0 {
        s += n % 10;
        n /= 10;
    }
    s
}

fn main() {
    for n in [123u64, 9999, 1, 0, 1024] {
        println!("digit_sum({}) = {}", n, digit_sum(n));
    }
}

```

Output:

```

digit_sum(123) = 6
digit_sum(9999) = 36
digit_sum(1) = 1
digit_sum(0) = 0
digit_sum(1024) = 7

```

44. Reverse the digits of a number

```
fn reverse_number(mut n: u64) -> u64 {
    let mut rev = 0u64;
    while n > 0 {
        rev = rev * 10 + n % 10;
        n /= 10;
    }
    rev
}

fn main() {
    for n in [12345u64, 100, 9, 8001] {
        println!("reverse({}) = {}", n, reverse_number(n));
    }
}
```

Output:

```
reverse(12345) = 54321
reverse(100) = 1
reverse(9) = 9
reverse(8001) = 1008
```

45. Check if a number is a palindrome

A number is a palindrome if it equals its reverse.

```
fn reverse_number(mut n: u64) -> u64 {
    let mut rev = 0u64;
    while n > 0 { rev = rev * 10 + n % 10; n /= 10; }
    rev
}

fn main() {
    for n in [121u64, 123, 1, 12321, 10] {
        let yes = reverse_number(n) == n;
        println!("{}", palindrome? {}, n, yes);
    }
}
```

Output:

```
121 palindrome? true
123 palindrome? false
1 palindrome? true
12321 palindrome? true
10 palindrome? false
```


46. Check if a number is an Armstrong number

An Armstrong number equals the sum of each digit raised to the count-of-digits power (e.g. $153 = 1^3 + 5^3 + 3^3$).

```
fn is_armstrong(n: u64) -> bool {
    let digits: Vec<u64> = n.to_string()
        .chars()
        .map(|c| c.to_digit(10).unwrap() as u64)
        .collect();
    let k = digits.len() as u32;
    let s: u64 = digits.iter().map(|d| d.pow(k)).sum();
    s == n
}

fn main() {
    for n in [153u64, 9474, 100, 9, 370] {
        println!("{}", Armstrong? {}, n, is_armstrong(n));
    }
}
```

Output:

```
153 Armstrong? true
9474 Armstrong? true
100 Armstrong? false
9 Armstrong? true
370 Armstrong? true
```

47. Print the first N Fibonacci numbers

Tuple assignment is idiomatic in Rust too.

```
fn main() {
    let n: usize = 10;
    let (mut a, mut b): (u64, u64) = (0, 1);
    for _ in 0..n {
        print!("{}", a);
        let next = a + b;
        a = b;
        b = next;
    }
    println!();
}
```

Output:

```
0 1 1 2 3 5 8 13 21 34
```

48. Swap two variables without a temporary

Rust supports destructuring assignment with `std::mem::swap` or a tuple pattern.

```
fn main() {
    let mut a = 10;
    let mut b = 20;
    println!("Before: a={}, b={}", a, b);

    // Method 1: std::mem::swap
    std::mem::swap(&mut a, &mut b);
    println!("After swap:      a={}, b={}", a, b);

    // Method 2: destructuring
    (a, b) = (b, a);
    println!("After destructure: a={}, b={}", a, b);
}
```

Output:

```
Before: a=10, b=20
After swap:      a=20, b=10
After destructure: a=10, b=20
```

49. Reverse a string

Use the iterator `rev()` method on chars.

```
fn main() {
    let s = "Hello, Rust!";
    let r: String = s.chars().rev().collect();
    println!("{}", r);
}
```

Output:

```
!tsuR ,olleH
```

50. Count vowels and consonants in a string

Use `chars()` with case-insensitive checks.

```
fn main() {
    let s = "Rust Programming Language";
    let mut v = 0;
    let mut c = 0;
    for ch in s.chars() {
        if ch.is_alphabetic() {
            let lo = ch.to_ascii_lowercase();
            if "aeiou".contains(lo) { v += 1; } else { c += 1; }
        }
    }
}
```

```

    }
    println!("Vowels: {}, Consonants: {}", v, c);
}

```

Output:

```
Vowels: 7, Consonants: 16
```

51. Sort a vector of numbers

Rust provides `sort()` (in-place, stable).

```

fn main() {
    let mut v = vec![5, 2, 9, 1, 7, 3, 8, 4, 6];
    v.sort();
    println!("Ascending : {:?}", v);
    v.sort_by(|a, b| b.cmp(a));
    println!("Descending: {:?}", v);
}

```

Output:

```

Ascending : [1, 2, 3, 4, 5, 6, 7, 8, 9]
Descending: [9, 8, 7, 6, 5, 4, 3, 2, 1]

```

52. Find the maximum and minimum in a vector

Iterators give us `.max()` and `.min()` directly.

```

fn main() {
    let v = vec![14, 7, 33, 2, 19, 88, 5];
    let max = v.iter().max().unwrap();
    let min = v.iter().min().unwrap();
    println!("Max = {}, Min = {}", max, min);
}

```

Output:

```
Max = 88, Min = 2
```

53. Linear search for an element in a vector

`position()` returns the first index where the predicate matches.

```

fn main() {
    let v = vec![10, 20, 30, 40, 50];
    let target = 30;
    match v.iter().position(|&x| x == target) {
        Some(i) => println!("Found {} at index {}", target, i),
        None    => println!("{}", target),
    }
}

```

```
}  
}
```

Output:

```
Found 30 at index 2
```

54. Bubble sort implementation

Classic $O(n^2)$ sort, useful as a teaching example.

```
fn bubble_sort(v: &mut Vec<i32>) {  
    let n = v.len();  
    for i in 0..n {  
        for j in 0..n - i - 1 {  
            if v[j] > v[j + 1] {  
                v.swap(j, j + 1);  
            }  
        }  
    }  
}  
  
fn main() {  
    let mut v = vec![64, 34, 25, 12, 22, 11, 90];  
    bubble_sort(&mut v);  
    println!("{:?}", v);  
}
```

Output:

```
[11, 12, 22, 25, 34, 64, 90]
```

55. Count the words in a string

`split_whitespace()` is the cleanest splitter.

```
fn main() {  
    let s = "Rust is blazingly fast and safe";  
    let n = s.split_whitespace().count();  
    println!("Word count: {}", n);  
}
```

Output:

```
Word count: 6
```

56. Count word frequencies using a HashMap

The `entry` API is the canonical way to upsert counts.

```
use std::collections::HashMap;

fn main() {
    let s = "the rat the cat the mat";
    let mut freq: HashMap<&str, u32> = HashMap::new();
    for w in s.split_whitespace() {
        *freq.entry(w).or_insert(0) += 1;
    }
    let mut items: Vec<_> = freq.iter().collect();
    items.sort();
    for (w, c) in items {
        println!("{}", w, c);
    }
}
```

Output:

```
cat: 1
mat: 1
rat: 1
the: 3
```

57. Read a text file line by line

Rust's `BufReader` gives buffered, line-by-line reads.

```
use std::fs::File;
use std::io::{self, BufRead, BufReader};

fn main() -> io::Result<()> {
    let file = File::open("data.txt")?;
    let reader = BufReader::new(file);
    for (i, line) in reader.lines().enumerate() {
        println!("{}", i + 1, line?);
    }
    Ok(())
}
```

Output:

```
1: First line
2: Second line
3: Third line
```

58. Write text to a file

Use `std::fs::write` for one-shot writes, or `OpenOptions + write_all` for appends.

```
use std::fs;
use std::io;

fn main() -> io::Result<()> {
```

```

    fs::write("output.txt", "Hello from Rust!\nLine 2\n")?;
    let back = fs::read_to_string("output.txt")?;
    print!("{}", back);
    Ok(())
}

```

Output:

```

Hello from Rust!
Line 2

```

59. Error handling with Result and ?

Rust uses `Result<T, E>` instead of exceptions. The `?` operator propagates errors.

```

use std::num::ParseIntError;

fn parse_and_double(s: &str) -> Result<i32, ParseIntError> {
    let n: i32 = s.parse()?;    // propagate on failure
    Ok(n * 2)
}

fn main() {
    for input in ["21", "abc", "42"] {
        match parse_and_double(input) {
            Ok(v) => println!("{}", input, v),
            Err(e) => println!("{}", input, e),
        }
    }
}

```

Output:

```

21 -> 42
abc -> error: invalid digit found in string
42 -> 84

```

60. Working with Option

Rust replaces None-like values with the explicit `Option` type. Pattern matching makes missing values impossible to ignore.

```

fn first_even(v: &[i32]) -> Option<i32> {
    v.iter().copied().find(|x| x % 2 == 0)
}

fn main() {
    let a = [1, 3, 6, 7, 8];
    let b = [1, 3, 5, 7];

    println!("{}", first_even(&a));    // Some(6)
    println!("{}", first_even(&b));    // None
}

```

```
// Common Option helpers:
println!("{}", first_even(&b).unwrap_or(-1)); // default
println!("{}", first_even(&a).map(|x| x * 10).unwrap_or(0));
}
```

Output:

```
Some(6)
None
-1
60
```

61. Iterator chains: map, filter, collect

Rust iterators are lazy and zero-cost — chain them freely.

```
fn main() {
    let nums = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

    // Square of even numbers
    let result: Vec<i32> = nums.iter()
        .filter(|&n| n % 2 == 0)
        .map(|&n| n * n)
        .collect();

    println!("{:?}", result);

    // Sum of those squares
    let total: i32 = result.iter().sum();
    println!("Sum = {}", total);
}
```

Output:

```
[4, 16, 36, 64, 100]
Sum = 220
```

62. Generic functions with trait bounds

Generics let you write code that works for any type satisfying given trait bounds.

```
use std::ops::Add;

fn add<T: Add<Output = T> + Copy>(a: T, b: T) -> T {
    a + b
}

fn main() {
    println!("{}", add(2, 3));
    println!("{}", add(2.5, 3.7));
    println!("{}", add(10u64, 20u64));
}
```

Output:

```
5
6.2
30
```

63. Trait objects (dynamic dispatch)

Use `Box<dyn Trait>` when you need a heterogeneous collection of things sharing a behavior.

```
trait Shape {
    fn area(&self) -> f64;
    fn name(&self) -> &str;
}

struct Circle { r: f64 }
struct Rectangle { w: f64, h: f64 }

impl Shape for Circle {
    fn area(&self) -> f64 { std::f64::consts::PI * self.r * self.r }
    fn name(&self) -> &str { "Circle" }
}

impl Shape for Rectangle {
    fn area(&self) -> f64 { self.w * self.h }
    fn name(&self) -> &str { "Rectangle" }
}

fn main() {
    let shapes: Vec<Box<dyn Shape>> = vec![
        Box::new(Circle { r: 3.0 }),
        Box::new(Rectangle { w: 4.0, h: 5.0 }),
    ];
    for s in &shapes {
        println!("{}", s.name(), s.area());
    }
}
```

Output:

```
Circle: 28.274
Rectangle: 20.000
```

64. Enums and exhaustive pattern matching

Rust's match must cover every variant — the compiler checks this.

```
#[derive(Debug)]
enum Traffic {
    Red,
    Yellow,
    Green,
```



```

}

fn action(t: &Traffic) -> &'static str {
    match t {
        Traffic::Red    => "Stop",
        Traffic::Yellow => "Slow down",
        Traffic::Green  => "Go",
    }
}

fn main() {
    for t in [Traffic::Red, Traffic::Yellow, Traffic::Green] {
        println!("{:?} -> {}", t, action(&t));
    }
}

```

Output:

```

Red -> Stop
Yellow -> Slow down
Green -> Go

```

65. Ownership, borrowing and references

Rust's core feature: each value has one owner. You can *borrow* values immutably (&T) or mutably (&mut T) without taking ownership.

```

fn length(s: &String) -> usize { s.len() }           // borrow
fn append_excl(s: &mut String) { s.push('!'); }      // mutable borrow

fn main() {
    let mut greeting = String::from("Hello");
    println!("Length: {}", length(&greeting)); // borrow read-only
    append_excl(&mut greeting);                // mutable borrow
    println!("{}", greeting);

    // Move semantics:
    let a = String::from("owned");
    let b = a; // ownership moves from a -> b
    // println!("{}", a); // <-- compile error: a is no longer valid
    println!("{}", b);
}

```

Output:

```

Length: 5
Hello!
owned

```

66. Slices on strings and vectors

A slice is a borrowed view into a contiguous sequence — no allocation.

```
fn first_word(s: &str) -> &str {
    s.split_whitespace().next().unwrap_or("")
}

fn main() {
    let s = String::from("Rust is awesome");
    println!("{}", first_word(&s));

    let v = vec![10, 20, 30, 40, 50];
    let mid: &[i32] = &v[1..4];
    println!("{:?}", mid);
}
```

Output:

```
Rust
[20, 30, 40]
```

67. Lifetimes — telling the compiler how long references live

Lifetimes are inferred most of the time, but you must annotate them when a function returns a reference derived from one of several inputs.

```
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() > b.len() { a } else { b }
}

fn main() {
    let s1 = String::from("short");
    let s2 = String::from("a longer string");
    println!("{}", longest(&s1, &s2));
}
```

Output:

```
a longer string
```

68. Constants and variable shadowing

Use `const` for compile-time values; redeclare a name with `let` to *shadow* the previous binding.

```
const MAX_RETRIES: u32 = 3;

fn main() {
    println!("MAX_RETRIES = {}", MAX_RETRIES);

    // Shadowing – note that the type can change.
```

```

let x = "5";
let x: i32 = x.parse().unwrap();
let x = x * 2;
println!("x = {}", x);
}

```

Output:

```

MAX_RETRIES = 3
x = 10

```

69. Tuple structs

When fields are positionally meaningful (e.g. RGB color), tuple structs are clearer than a regular struct.

```

struct Color(u8, u8, u8);

impl Color {
    fn hex(&self) -> String {
        format!("#{:02X}{:02X}{:02X}", self.0, self.1, self.2)
    }
}

fn main() {
    let red = Color(255, 0, 0);
    let teal = Color(0, 128, 128);
    println!("Red -> {}", red.hex());
    println!("Teal -> {}", teal.hex());
}

```

Output:

```

Red -> #FF0000
Teal -> #008080

```

70. Closures capturing variables from the environment

Closures automatically capture by reference, by mutable reference, or by move — Rust picks the least restrictive that compiles.

```

fn main() {
    let multiplier = 5;
    let times = |x: i32| x * multiplier; // captures multiplier by reference
    println!("{}", times(3));

    let mut counter = 0;
    let mut tick = || { counter += 1; counter }; // captures by mut ref
    println!("{}", tick(), tick(), tick());

    let owned = String::from("hello");
    let print_it = move || println!("{}", owned); // by move
    print_it();
}

```

```
}
```

Output:

```
15
1 2 3
hello
```

71. A note on Cargo, modules and crates

Rust projects are built with **Cargo**. A typical workflow:

```
# Create a new binary project
$ cargo new my_project
$ cd my_project

# Source layout:
#   src/main.rs    <-- binary entry point
#   Cargo.toml     <-- dependencies & metadata

# Add a dependency
$ cargo add rand

# Run
$ cargo run

# Run optimized release build
$ cargo build --release
$ ./target/release/my_project
```

Note: Cargo handles compilation, testing, formatting and dependency resolution all in one tool.

Compiled by Rejwar

<https://www.linkedin.com/in/rejwar/>

Connect for more Rust and systems-programming content.